
Aurum — Elite B2B/B2C Talent Platform

A premium professional talent placement and services marketplace connecting businesses with elite global talent. Built with a luxury noir/gold aesthetic, optimised for conversion.

Live Demo

Deployed on Replit — accessible via the published .replit.app domain.

Table of Contents

- Project Overview
- Features
- Tech Stack
- Architecture
- Project Structure
- Database Schema
- API Reference
- Pages & Routes
- Admin Access
- Calendly Integration
- Environment Variables
- Development Setup
- Deployment

Project Overview

Aurum is a full-stack B2B/B2C web platform built to:

- Present elite professional services with a high-converting landing page
- Capture leads via a contact form and Calendly booking integration
- Manage leads through a CRM pipeline (Kanban board with 7 stages)
- Maintain a filterable candidate database with matching system
- Provide clients a self-service portal to track their placement status

Built per a detailed product specification (cahier des charges) covering both MVP public-facing pages and a full V2 back-office.

Features

Public Front-Office

- Landing Page — Hero, social proof, service profile cards, step-by-step process, differentiators, testimonials, CTA
- Services Catalogue — Filterable grid of service profiles
- Book a Call — Calendly inline booking widget + contact request form
- FAQ — Accordion-style frequently asked questions
- Legal — GDPR notice and Terms & Conditions placeholder
- Client Portal — Clients enter their Contact ID to view pipeline status and matched candidates

Admin Back-Office (password protected)

- Dashboard — KPI cards (leads, candidates, matches, conversion rate, pipeline value), bar chart by stage, recent leads table
- CRM Pipeline — Kanban board with 7 stages: New Lead !' Contacted !' Qualified !' Proposal Sent !' Negotiation !' Closed Won !' Closed Lost. Move leads between stages, add notes and deal values.
- Candidate Database — Full CRUD with filters by role, language, experience, rate, and status
- Matching System — Assign candidates to client needs with a match score and status tracking

Tech Stack

Layer	Technology
Frontend	React 18, Vite, TypeScript
Styling	Tailwind CSS, shadcn/ui
Routing	wouter
Forms	react-hook-form + zod
Animations	Framer Motion
Charts	Recharts
Backend	Node.js, Express 5, TypeScript
Database	PostgreSQL
ORM	Drizzle ORM
Validation	Zod v4, drizzle-zod
API Codegen	Orval (OpenAPI !' React Query hooks + Zod schemas)
State/Fetching	TanStack Query (React Query)
Package Manager	pnpm (workspaces monorepo)
Build Tool	esbuild
Hosting	Replit (frontend + backend + database)

Architecture

This is a pnpm monorepo with path-based routing through a shared reverse proxy.

```
Browser !' Replit Proxy (localhost:80)
      % % % /      !' artifacts/platform (React/Vite frontend)
      % % % /api    !' artifacts/api-server (Express API)
```

Monorepo Packages

```
workspace/
% % % artifacts/
%   % % % platform/           # React + Vite frontend
%   % % % api-server/         # Express 5 API server
% % % lib/
%   % % % api-spec/           # OpenAPI YAML + Orval codegen config
%   % % % api-client-react/    # Generated React Query hooks (DO NOT EDIT)
%   % % % api-zod/             # Generated Zod validators (DO NOT EDIT)
%   % % % db/                  # Drizzle ORM schema + client
% % % scripts/                # Shared utility scripts
```

Code Generation Flow

```
lib/api-spec/openapi.yaml
!"" (pnpm --filter @workspace/api-spec run codegen)
lib/api-client-react/  !• React Query hooks (auto-generated)
lib/api-zod/           !• Zod schemas (auto-generated)
```

The frontend imports hooks from @workspace/api-client-react. The backend imports validators from @workspace/api-zod. Both are generated from the single OpenAPI source of truth.

Project Structure

```
artifacts/platform/src/
% % % components/
%   % % % layout.tsx          # Main navbar + footer wrapper
%   % % % admin-layout.tsx    # Admin sidebar layout
%   % % % admin-auth.tsx     # Admin password gate (context + login screen)
%   % % % ui/                  # shadcn/ui components
% % % pages/
%   % % % home.tsx            # Landing page
%   % % % services.tsx        # Services catalogue
%   % % % contact.tsx         # Book a call (Calendly + form)
%   % % % faq.tsx             # FAQ accordion
%   % % % legal.tsx           # Legal / GDPR
%   % % % not-found.tsx       # 404
%   % % % admin/
%   %   % % % dashboard.tsx    # Admin dashboard
%   %   % % % crm.tsx          # CRM Kanban pipeline
%   %   % % % candidates.tsx   # Candidate database
%   %   % % % matching.tsx     # Matching system
%   % % % client/
%     % % % portal.tsx        # Client self-service portal
...

```

Database Schema

contacts	Type	Description
id	serial PK	Auto-increment
name	text	Contact full name
company	text	Company name
email	text	Work email
need	text	Requirements description
created_at	timestamp	Creation date

candidates

Column	Type	Description
id	serial PK	Auto-increment
name	text	Candidate full name
role	text	Job title / role
skills	text[]	Array of skills
language	text	Primary language
experience_years	integer	Years of experience
hourly_rate	integer	Rate in €/hr
availability	text	available / on_assignment / etc.
status	text	active / inactive
bio	text (nullable)	Short biography
location	text (nullable)	Location
created_at	timestamp	Creation date

Column	Type	Description
id	serial PK	Auto-increment
contact_id	integer FK !' contacts	Associated contact
stage	text	Pipeline stage
notes	text (nullable)	Internal notes
value	integer	Estimated deal value (€)
created_at	timestamp	Creation date
updated_at	timestamp	Last update

Pipeline stages: new !' contacted !' qualified !' proposal !' negotiation !' closed_won !' closed_lost

Column	Type	Description
id	serial PK	Auto-increment
contact_id	integer FK !' contacts	Client
candidate_id	integer FK !' candidates	Matched candidate
score	integer	Match score (0–100)
status	text	proposed / accepted / rejected
notes	text (nullable)	Match notes
created_at	timestamp	Creation date

API Reference

Base URL: /api

Method	Endpoint	Description
GET	/healthz	Server health check

Method	Endpoint	Description
POST	/contact	Submit a contact / book a call request
GET	/contact/stats	Lead statistics (total, this week, this month)
GET	/contact/list	List all contacts/leads

Candidates

Method	Endpoint	Description
GET	/candidates	List candidates (filters: language, role, minExperience, maxDate, status)
POST	/candidates	Create a new candidate
GET	/candidates/:id	Get a candidate by ID
PATCH	/candidates/:id	Update a candidate
DELETE	/candidates/:id	Delete a candidate

Method	Endpoint	CRM Pipeline
GET	/pipeline	Full pipeline with leads grouped by stage + totals
POST	/pipeline/leads	Add a contact to the pipeline
PATCH	/pipeline/leads/:id	Move a lead to a new stage / update notes/value
DELETE	/pipeline/leads/:id	Remove a lead from the pipeline

Method	Endpoint	Matching
GET	/matching?contactId=X	Get all matches for a given contact
POST	/matching	Create a candidate–contact match

Method	Endpoint	Admin
GET	/admin/stats	Full analytics: KPIs, leads by stage, recent leads

Method	Endpoint	Client Portal
GET	/client/:contactId	Client portal data — stage, status, messages, matched candidates

Pages & Routes

Path	Description	Access
/	Landing page	Public
/services	Services catalogue	Public
/contact	Book a call (Calendly + form)	Public
/faq	FAQ accordion	Public
/legal	GDPR / Terms	Public
/client	Client self-service portal	Public (requires Contact ID)
/admin	Admin dashboard	Password protected
/admin/crm	CRM Kanban pipeline	Password protected
/admin/candidates	Candidate database	Password protected
/admin/matching	Matching system	Password protected

Admin Access

All `/admin/*` routes are protected by a password gate.

The password is read from the `VITE_ADMIN_PASSWORD` environment secret (see Environment Variables). Set it in Replit's Secrets panel and redeploy to change it.

Session is stored in sessionStorage (cleared when the browser tab is closed). A "Logout" button is available in the admin sidebar.

Calendly Integration

The /contact page embeds your Calendly booking widget directly.

Current link: <https://calendly.com/luca-adam23/30min>

To update the link, edit the CALENDLY_URL constant in artifacts/platform/src/pages/contact.tsx:

```
const CALENDLY_URL = "https://calendly.com/your-username/your-event";
```

The widget is embedded as an iframe styled to match the Aurum dark theme (background_color=0d0d0d, text_color=ffffff, primary_color=c9a84c).

Environment Variables

Variable	Where	Description
DATABASE_URL	API Server	PostgreSQL connection string (auto-set by Replit)
SESSION_SECRET	API Server	Secret for session signing
PORT	Both	Port assigned by Replit proxy (auto-set)
VITE_ADMIN_PASSWORD	Frontend (Platform)	Password for the admin panel — store as a Replit Secret, injected at build time via Vite

Development Setup

Prerequisites

- Node.js 20+
- pnpm 9+
- PostgreSQL database (or use Replit's built-in DB)

Installation

```
# Clone the repository
git clone https://github.com/your-username/aurum-platform.git
cd aurum-platform

# Install all dependencies
pnpm install

# Push database schema
pnpm --filter @workspace/db run push

# Run codegen (if you modified the OpenAPI spec)
pnpm --filter @workspace/api-spec run codegen
```

Running Locally

Start both services in separate terminals:

```
# Terminal 1 – API server
pnpm --filter @workspace/api-server run dev
```

```
# Terminal 2 – Frontend
pnpm --filter @workspace/platform run dev
```

Or start both via Replit workflows.

Useful Commands

```
# Full typecheck
pnpm run typecheck
```

```
# Build API server
pnpm --filter @workspace/api-server run build
```

```
# Regenerate API client from OpenAPI spec
pnpm --filter @workspace/api-spec run codegen
```

```
# Push DB schema changes (development only – destructive)
pnpm --filter @workspace/db run push
```

Adding a New API Endpoint

Edit `lib/api-spec/openapi.yaml` — add your path, request/response schemas

Run `pnpm --filter @workspace/api-spec run codegen` — generates hooks + Zod validators

Add the Express route in `artifacts/api-server/src/routes/`

Register it in `artifacts/api-server/src/routes/index.ts`

Use the generated hook in the frontend via `@workspace/api-client-react`

Deployment

The application is deployed on Replit and published via the built-in deployment system.

- Frontend is served at the root path /
- API is served at /api
- Both are routed through Replit's shared reverse proxy
- The PostgreSQL database is managed by Replit and auto-connected via `DATABASE_URL`

To redeploy after changes: click "Publish" in the Replit interface, or use the Replit CLI.

KPIs (as per specification)

The platform is instrumented to track:

- Conversion rate — `closed_won` leads / total leads (visible in admin dashboard)
- Calls booked — total contacts in database
- Pipeline value — sum of deal values across all pipeline stages
- Leads by stage — bar chart in admin dashboard

Roadmap (from specification)

- Authentication system for clients (SSO / magic link)
- Email notifications on new contact form submission

- Stripe payment integration for future billing
- HubSpot CRM sync
- Advanced AI matching algorithm (skills similarity scoring)
- Admin password stored as environment variable (not hardcoded)
- Google Calendar integration alongside Calendly

